

곤지암리조트 식음업장 수요예측

팀 브리핑 — 현재 지점, 핵심 쟁점, 다음 할 일

작성 2026-08-01 현재 성적 Public LB 0.4818 · 55등 통과선 0.711046 출처 experiments/log.md + 실험 JSON 7건 + src/

이 문서를 3줄로

- ① 예선 통과선(0.711)은 이미 여유 있게 넘겼다. 지금 싸움은 **55등 → 상위권**이다.
- ② 지난 3일의 진짜 성과는 점수가 아니라 **측정 도구**다 — 공식 지표를 직접 구현했고, 우리 실험의 **노이즈 바닥이 0.003**이라는 걸 알아냈다. 이걸 모르고 0.002짜리 개선을 쫓으면 헛수고다.
- ③ 지금까지 **피쳐만** 손냈는데, 남은 기대값은 **하이퍼파라미터 · 알고리즘 앙상블** 쪽이 더 크다. 다음 작업은 여기부터다.

1. 배경 — 이 대회가 무엇인가

1.1 무엇을 맞추는가 — 그리고 "창(window)"이라는 말

곤지암리조트의 식음업장 9곳 × 메뉴 193개에 대해, 28일치 판매 기록을 주면 그 다음 +1일 ~ +7일의 일별 판매수량을 맞추는 문제다.

주어지는 것 — 관측 28일의 일별 판매수량 (품목 193개 전부)

맞힐 것
+1 ~ +7일

이 28일 덩어리를 "창(window)"이라고 부른다

7일 × 193품목
= 1,351칸

용어 정리 — 이 문서 전체에서 계속 나온다

창(window) = 우리에게 주어지는 28일짜리 관측 구간. 이 안의 정보만 써서 예측해야 한다. 대회 규칙상 창 밖(작년 같은 날 등)을 들여다보는 건 금지다.

origin = 창 **의** 마지막 날. "지금"에 해당하는 날이다. 타깃은 항상 $\text{origin}+1 \sim \text{origin}+7$.

horizon = origin으로부터 며칠 뒤인가(1~7). 당연히 멀수록 어렵다.

중요한 건 "창 안의 28일이 정보의 전부"라는 점이다. 예를 들어 w_mean (창 평균)이라는 피쳐는 이 28칸의 평균이라는 뜻이고, d_mean (같은 요일 평균)은 28일 안에 들어 있는 같은 요일 4개의 평균이라는 뜻이다. 뒤에 나오는 "창 통계"는 전부 이 28칸을 이리저리 요약한 숫자들이다.

학습할 때는 이 창을 과거 데이터 위에서 하루씩 미끄러뜨리며 샘플을 만든다. 2023-01-28을 origin으로 한 샘플, 2023-01-29를 origin으로 한 샘플... 이렇게 origin 497개 × 7일 × 193품목 ≈ **67만 개**의 학습 샘플이 나온다 (여기서 $y=0$ 인 것을 빼면 약 32만 개).

1.2 데이터 생김새

항목	내용
학습 데이터	train.csv · 2023-01-01 ~ 2024-06-15(약 1.5년) · 102,676행 · long 포맷(영업일자 / 영업장명_메뉴명 / 매출수량)
테스트	TEST_00 ~ TEST_09 10개. 각각 독립된 28일 창만 주어짐
제출 형식	70행 × 194열 (10개 테스트 × 7일 = 70행, 품목 193개 + 식별자 1열)
데이터 성격	값의 52.6%가 0 · 최댓값 1,372 · 음수(환불) 14행 존재

테스트 10개의 예측 구간은 서로 이어져서 2024-06-16 ~ 2025-05-31 한 해를 빈틈없이 덮는다. 즉 사계절이 전부 채점 대상이다. 그중 셋은 계절 전환 주간이라 특히 어렵다 — TEST_04(12/1~7, 화담숲 소등) · TEST_07(3/16~22, 스키 폐장) · TEST_08(4/20~26, 화담숲 점등).

1.3 영업장 9곳의 성격 — 성수기 방향이 서로 다르다

웹 리서치로 세우고 데이터로 검증한 5개 군집이다. "리조트 전체가 언제 붐비는가"라는 단일 시즌 개념이 통하지 않는다는 게 핵심이다.

군집	영업장	성수기 / 특징
화담숲 연동	화담숲주막 · 화담숲카페	12·1·2월 판매량이 정확히 0(완전 휴장). 피크는 2023-10 단풍철
스키·겨울	포레스트릿 · 카페테리아	슬로프 옆 스넥 / 스키하우스 푸드코트. 1월 피크
야외 그린시즌	느티나무 셀프BBQ	6월 피크. 리서치 추정과 달리 겨울에 닫지는 않는다(데이터로 확인)
상시	담하 · 라그로타 · 미라시아	연중 영업. 라그로타는 월요일 휴무. 담하·미라시아는 채점 가중치가 높다
B2B	연회장	컨벤션. 목요일 최고 · 일요일 최저 — 다른 곳과 요일 패턴이 정반대

참고: 3월 초는 스키 종료와 화담숲 개장 사이의 틈이라 9곳 중 6곳이 연중 최저를 찍는다.

1.4 채점 방법 — SMAPE

칸 하나의 벌점은 이렇게 계산된다. A는 실측, P는 예측이다.

$$SMAPE(A, P) = \frac{2|A - P|}{|A| + |P|}$$

직관: 절대오차가 아니라 비율오차다. 100개 팔리는 메뉴를 10개 틀리는 것보다, 2개 팔리는 메뉴를 2개 틀리는 게 훨씬 큰 벌점이다. 0(완벽)에서 2(최악) 사이 값이다.

실측 A	예측 P	벌점	해석
10	10	0.000	정답
10	5	0.667	절반으로 과소예측
10	20	0.667	두 배로 과대예측 — 배수 기준으로 대칭
10	0	2.000	최대 벌점
100	90	0.105	10을 틀렸는데도 벌점은 작다
2	4	0.667	2개 틀렸는데 위의 6배 벌점 — 작은 품목이 무섭다

1.5 대회 규칙의 제약

- 외부 데이터 · 크롤링 · API 전면 금지. 날씨 데이터를 쓸 수 없다. 단 "공휴일 · 요일" 같은 도메인 지식은 허용된다.
- 평가 샘플은 서로 독립 추론. 다른 TEST 파일을 참조할 수 없고, 각자에게 주어진 **28일** **창만** 써야 한다.
- 평가 Input/Target을 학습에 쓰는 것, pseudo labeling 금지.

이 조항 때문에 초기에 시도했던 "작년 동기(YoY)" 피쳐는 성능이 없었을 뿐 아니라 **규칙 위반**이기도 했다(28일 창 밖을 조회).

2. 지금까지 한 것과 알아낸 것

2.1 성적 사다리

점수는 **낮을수록 좋다**(0~2 스케일). 공식 지표 기준이다.

전부 0으로 예측	2.000	
전부 1로 예측	1.117	
상수 최적(전부 3)	0.938	
통과선 (Public)	0.711	
규칙 베이스라인	0.619	
LGBM v1 (CV 3폴드)	0.517	
LGBM v1 (실제 제출 LB)	0.482	
현재 1등	0.440	

규칙 베이스라인이란 머신러닝 없이 "**창 안에서 같은 요일의 양수 평균을 그대로 답으로 낸다**"는 세 줄짜리 규칙이다. 예를 들어 다음 주 토요일을 예측할 때, 창 안에 있는 토요일 4개 중 0이 아닌 것들의 평균을 낸다. 이것만으로도 통과선을 넘는다. **모델의 순수 기여는 0.619 → 0.482 구간**이다.

2.2 현재 파이프라인 구성

요소	내용
모델	글로벌 LightGBM 1개 (전 품목 공통) · num_leaves=255 · rounds=1000 · 시드 5개 앙상블
타깃	log1p 변환 + L1 목적함수. y=0인 행은 학습에서 제거
피쳐	57개 — 4장에서 전부 설명
후처리	예측 하한 1.0
검증	롤링-오리진 3폴드(가을 / 겨울 / 봄) — 쟁점 ④에서 설명

"**글로벌 모델 1개**"란 품목마다 모델을 따로 만들지 않고, 193개 품목의 데이터를 전부 한 덩어리로 합쳐 **모델 하나**를 학습시켰다는 뜻이다. 품목 구분은 `item_id` 라는 피쳐로 알려준다. 품목별로 쪼개면 각 모델의 학습 데이터가 1/193로 줄어드는데, 그러면 희귀 품목은 학습 자체가 안 된다.

2.3 확정된 결론 — 각각 "무엇을 한 것인가"

지금까지 실제로 **손을 댄 것들**이다. 각각 무엇을 왜 했고 얼마나 좋아졌는지 적는다.

① 학습에서 $y=0$ 인 행을 제거 — 개선 0.352 (단일 최대)

무엇을: 학습 샘플 67만 개 중 실제 판매량이 0인 행을 통째로 버리고 나머지 32만 개로만 학습시켰다.

왜: 채점에서 0인 칸이 제외되니, "안 팔릴 날을 맞히는 능력"에는 점수가 걸려 있지 않다. 그런데 데이터의 절반이 0이라 그냥 학습시키면 모델이 절반의 노력을 점수와 무관한 일에 쓴다.

결과: 0.895 → 0.543. 자세한 논리는 쟁점 ①.

② 타깃을 $\log(1+y)$ 로 바꾸고 L1 손실 사용 — 채택

무엇을: 판매량 y 를 그대로 맞히는 대신 $\log(1+y)$ 를 맞히도록 학습시켰다. 오차 함수는 제곱(L2)이 아니라 절댓값(L1)을 썼다.

왜: 판매량이 1부터 1,372까지 퍼져 있다. 그대로 학습시키면 큰 값에서 나는 오차가 손실을 지배해서 모델이 작은 품목을 대충 맞히게 된다. 로그를 씌우면 "몇 배 틀렸나"가 균등해져서 비율오차인 SMAPE와 결이 맞는다. L1은 이상치에 덜 흔들린다.

결과: $l2 \cdot \text{huber}$ 목적함수보다 나았다.

③ 예측 하한 1.0 — 개선 0.004, 증명 있음

무엇을: 모델이 뱉은 예측값이 1보다 작으면 전부 1로 올렸다.

왜: 채점 대상은 전부 $|A| \geq 1$ 이고 수량은 정수다. 따라서 $p < 1$ 구간에서 벌점 $2(A-P)/(A+P)$ 는 P 가 커질수록 단조감소한다 — 즉 1 미만 예측은 어떤 실측값에 대해서도 1로 올린 것보다 나쁘다. 손해 볼 수 없는 개선이다.

결과: 하한 0 → 0.5953 · 하한 1.0 → 0.5914 · 하한 1.5 → 0.6037 · 하한 2.0 → 0.6093. 1을 넘기면 실측 1인 21% 행을 과대예측하기 시작해 손해로 돌아선다.

주의: "하한 1"과 "전부 1로 예측"은 완전히 다르다. 후자는 1.117로 규칙 베이스라인의 두 배 가까이 나쁘다. 그리고 규칙·단순 모델일수록 이 이득이 크다 — LightGBM은 애초에 1 미만을 거의 안 뱉어서(최소 0.80) 초기 실험에서는 효과가 가려져 있었다.

④ 시드 5개 앙상블 — 개선 0.005

무엇을: 완전히 똑같은 설정으로 랜덤 시드만 다르게 모델 5개를 만들어, 예측값을 평균냈다.

왜: LightGBM은 학습할 때 데이터와 피처를 랜덤하게 일부만 뽑아 쓴다(bagging_fraction, feature_fraction). 그래서 시드가 다르면 다른 모델이 나온다. 각 모델의 개별 실수는 방향이 랜덤이라 평균내면 상쇄된다.

결과: +0.005. 부수효과로 측정 노이즈도 줄어든다(쟁점 ③에서 자세히).

⑤ 트리를 크게 leaves 255 · rounds 1000 — 개선 0.004

무엇을: 트리 하나가 가질 수 있는 잎(최종 분기)의 개수를 255개로, 트리 개수를 1,000개로 늘렸다.

왜: 학습 행이 32만 줄이라 LightGBM 기본값(잎 31개)은 너무 단순하다. 표현력이 모자라면 품목 × 요일 × 계절 같은 조합을 못 잡는다.

결과: +0.004. 다만 하이퍼파라미터 중 건드린 건 이 두 개가 전부다 — 그래서 다음 할 일 1순위가 여기서다.

⑥ 휴점일 보정 피처 6개 — 개선 0.0036

무엇을: 창 28일 중 그 영업장이 문을 닫았던 날을 찾아내서(영업장 전체 합계가 0인 날 = 휴점), 그 날들을 빼고 계산한 평균을 별도 피처로 넣었다.

왜: 화담숲 46% · 포레스트릿 28% · 라그로타 13%가 휴점일이다. 창 28일 중 절반이 휴점이면 w_{mean} (창 평균)이 반토막이 난다. 모델은 이것 "수요가 줄었다"로 오해한다. 실제로는 영업일수가 줄었을 뿐인데.

결과: +0.0036. 세 폴드 전부 개선.

⑦ 가격을 5등급으로 묶기 — ⚠️ 입증 안 됨

무엇을: 웹 리서치로 모은 메뉴 가격을 3천 / 8천 / 2만 / 5만원 경계로 5등급 번호로 바꿔 넣었다.

왜: 원본 숫자를 그대로 주면 트리가 임의의 지점에서 쪼갬다. 묶어주면 "저가/고가"라는 의미 단위로 학습한다.

(가격을 아예 빼면 오히려 나빠지므로 정보 자체는 있다.)
결과: $-0.0012 - 2\sigma$ 를 못 넘어서 입증되지 않았다(쟁점 ③). 게다가 코드와 로그가 서로 다르게 기록돼 있어 미팅 안건이다.

2.4 효과가 없었던 것 — 재시도 금지 목록

시도	무엇을 한 것인가	왜 안 됐나
YoY(작년 동기) 피쳐	작년 같은 날짜의 판매량을 피쳐로	학습 기간이 1.5년뿐이라 대부분 NaN. 게다가 창 밖 조회라 규칙 위반
최신 데이터 가중	최근 샘플에 큰 학습 가중치	차이 없음. 창 통계가 이미 최근을 반영
l2 / huber 목적함수	손실 함수 교체	L1보다 나쁨. 큰 값에 끌려간다
예측 일괄 보정계수	최종 예측에 $\times 0.95$ 같은 상수를 곱함	차이 없음
대형 스파이크 캡핑	상위 1%를 99분위 값으로 잘라냄(3안)	3안 전부 나빠짐. SMAPE가 비율오차라 큰 값의 절대 오차가 그대로 별점이 되지 않는다. 캡핑하면 진짜 큰 날만 놓쳐 손해
품목별 요일 프로파일	학습기간 전체로 품목별 요일 성향을 만들어 피쳐로	모델이 item_id $\times$ dow로 이미 만들 수 있던 정보. 4.4절 참고
검증을 촘촘하게	검증 시점 간격 7일 $\rightarrow$ 1일	노이즈가 전혀 안 줄고 시간만 6배. 쟁점 ③ 참고
음수(환불) 처리	-값을 어떻게 다룰지 실험 예정이었음	프로파일링해보니 14행(0.014%)뿐. 점수 기여 상한 0.0006으로 썰 가치가 없어 취소

2.5 채점 대상 y의 분포

0인 행이 빠지고 나면 남는 것들의 분포다(검증 폴드 29,155행).

y=1	y=2	3~5	6~20	21~100	y>100	중앙값	평균
21.2%	13.1%	20.7%	22.9%	17.1%	5.0%	4	22.8

상수로 찍었을 때 최적은 3(0.938)인데 중앙값은 4다. SMAPE는 "중앙보다 살짝 아래"를 선호한다는 성질이 여기서 확인된다. 위 1.4절 표에서 봤듯 과대예측이 과소예측보다 살짝 더 아프기 때문이다.

2.6 오차가 어디에 남아 있는가

오차 소재	SMAPE	성격
화담숲카페	0.688	계절 매장 — 개·폐장 전환이 어렵다
포레스트릿	0.655	
화담숲주막	0.605	
담하	0.556	가중치가 높은 업장이라 실제 손해는 더 크다
y > 100 대형건	0.702	단체·행사성 수요
먼 horizon	+1일 0.498 $\rightarrow$ +6일 0.577	날짜가 멀수록 벌어진다
연회장 (가장 잘 맞음)	0.356	B2B라 패턴이 규칙적

3. 이 문제의 핵심 쟁점 넷

여기부터가 이 프로젝트에서 실제로 배운 것들이다. 전부 방법론에 관한 것이고, 그게 점수보다 중요하다.

쟁점 ① 평가식이 전략을 정한다 — "팔릴까"가 아니라 "팔린다면 얼마"

대회 규칙에 이 한 줄이 있다: "실제 매출 수량이 0인 경우 평가 산식에 반영되지 않음."
데이터의 52.6%가 0이니, 처음엔 자연스럽게 이렇게 생각했다 — "수요예측 문제니까 ㉠오늘 팔릴까? ㉢팔린다면 얼마? 두 단계로 풀어야겠다(hurdle 모델)." 실제로 SMAPE는 0을 많이 찍으면 유리해 보이기도 한다.

그런데 그게 함정이다

실측이 0인 칸은 채점에서 아예 빠진다. 그러니 "안 팔릴 날을 맞히는 것"에는 점수가 1점도 걸려 있지 않다. 반대로 전부 0으로 예측하면 채점 대상(전부 A≠0)에서 모조리 최대 벌점을 맞아 2.000, 최악이 된다.
→ ㉠단계는 통째로 필요 없다. 문제는 "팔린다면 얼마" 하나뿐이다.

그래서 학습 데이터에서 y=0인 행을 빼버렸다. 같은 LightGBM, 같은 피쳐인데 0.895 → 0.543. 이 프로젝트 전체에서 단일 최대 개선이며, 모델을 건드려서 얻은 게 아니라 평가식을 정확히 읽어서 얻었다.

쟁점 ② 지표의 구조가 우선순위를 정한다 — 희귀 품목도 주력 품목과 1표씩

공식 점수는 단순 평균이 아니라 3중 평균이다.

Score = \sum_s w_s \left(\frac{1}{|I_s|} \sum_{i \in I_s} \left(\frac{1}{T_i} \sum_t \frac{2|A_{t,i} - P_{t,i}|}{|A_{t,i}| + |P_{t,i}|} \right) \right)

- 1. 품목 안에서 — 실측이 0이 아닌 날짜들끼리 평균 (1/T_i)
- 2. 업장 안에서 — 품목끼리 단순평균 (1/|I_s|) ← 여기가 핵심
- 3. 업장끼리 — 가중합 (w_s, 담하·미라시아가 높으나 값은 비공개)

숫자로 보는 예시 — 담하에 품목이 2개뿐이라고 치자

실제 담하에는 42개 품목이 있지만, 원리는 2개로 충분히 보인다.

품목	채점되는 날짜 수	그 날짜들의 평균 벌점	성격
공깃밥	20일	0.30	거의 매일 팔리는 주력. 예측이 쉽다
희귀 코스요리	2일	1.20	어쩌다 한 번. 예측이 어렵다

채점 방식	계산	담하 점수
flat (행 단위 평균) 우리가 처음 쓰던 방식	(20×0.30 + 2×1.20) ÷ 22 = (6.0 + 2.4) ÷ 22	0.382
공식 (품목 단위 평균)	(0.30 + 1.20) ÷ 2	0.750

거의 두 배 차이가 난다. flat에서는 공깃밥이 20표, 코스요리가 2표를 행사해서 쉬운 품목이 점수를 지배한다. 공식 지표에서는 둘 다 정확히 1표씩이라 어려운 품목이 그대로 드러난다.

그래서 "무엇을 개선해야 하는가"가 달라진다

모델을 고쳐서 희귀 코스요리의 벌점만 1.20 → 0.90으로 낮췄다고 하자. 주력 품목은 그대로다.

채점 방식	개선 전	개선 후	개선폭	체감
flat	0.382	0.355	0.027	"별거 아니네"
공식	0.750	0.600	0.150	"엄청난 개선이다"

같은 개선인데 공식 지표에서 5.5배 크게 보인다

즉 어떤 자료 재느냐에 따라 "어디에 노력을 쏟아야 하는가"의 답이 바뀐다. flat으로 재면 주력 품목만 신경 쓰게 되고, 공식으로 재면 희귀 품목을 잡는 게 훨씬 값어치 있는 일이 된다.

실제로 우리 실험에서도 결론 두 개가 뒤집혔다:

- 규칙 대비 모델의 이득이 flat 0.053 → 공식 0.102로 두 배가 됐다. 모델이 바로 그 희귀 품목에서 규칙을 크게 이기기 때문이다 — 규칙은 "창 안 같은 요일 평균"이라 표본이 4개뿐인 희귀 품목에서 무력하다.
- 규칙과 모델을 섞는 최적 비율이 flat에서 9:1 → 공식에서 10:0(모델 단독)으로 바뀌었다. 잘못된 지표로 튜닝했으면 쓸데없이 규칙을 섞고 있었을 것이다.

가중치 w_s 는 몰라도 된다. 민감도를 재보니 균등 0.6185 / 1.5배 0.6155 / 2배 0.6131 / 3배 0.6093으로 폭이 0.009밖에 안 된다. 모델 간 우열이 뒤집힐 가능성이 낮아 가장 보수적인 균등 가중을 기본 지표로 쓰고, 최종 판단 때만 민감도를 병기한다.

쟁점 ③ 측정 한계를 먼저 알아야 한다 — 우리 저울의 눈금은 0.003이다

이 프로젝트에서 가장 중요한 방법론 발견이다.

완전히 똑같은 설정에서 랜덤 시드만 바꿔 5번 돌려봤다. 코드도, 피쳐도, 데이터도 전부 동일하다. 바꾼 건 난수 시드 하나뿐이다.

1회	2회	3회	4회	5회	표준편차 σ	2σ
0.5159	0.5183	0.5173	0.5184	0.5201	0.0014	0.0028

판독 기준

점수 차이가 0.003 미만이면 우연으로 간주한다. 아무것도 안 바뀌도 그 정도는 흔들리기 때문이다.

이 기준을 적용하자 앞서 내렸던 결론 대부분이 근거 없음으로 판명됐다. 우리는 노이즈를 신호로 읽고 있었다.

앞서 내렸던 결론	실제 차이	재판정
도메인 플래그가 잉여다	+0.0016	철회
dow 그룹이 중복이다	+0.0008	근거 없음
ctx 기여가 미미하다	+0.0004	근거 없음
가격 5등급 범주화가 이득이다	-0.0012	입증 안 됨
휴점일 보정 피쳐가 이득이다	-0.0028	경계선 → 이후 재측정으로 확정

노이즈를 줄이려 한 시도 두 번 — 둘 다 실패했다

시도 1: 검증 표본을 7배로 늘린다. 검증 시점을 7일 간격 → 1일 간격으로 촘촘하게 했다(42개 → 289개, 채점행 2.9만 → 20만). 결과는 σ 0.0014 → 0.0015. 전혀 안 줄었고 실행시간만 85초 → 470초가 됐다.

왜 실패했나 — 노이즈의 출처를 잘못 짚었다

표본을 늘리면 줄어드는 건 표본에서 오는 노이즈일 때다. 여기 노이즈는 모델(시드)에서 온다. 시드가 바뀌면 다른 트리가 만들어지고, 그 모델은 특정 품목을 모든 날짜에서 체계적으로 다르게 예측한다. 날짜를 더 넣어도 같은 편향이 반복될 뿐 상쇄되지 않는다. 게다가 인접한 날짜는 예측 구간이 6일이나 겹쳐서 새 정보가 거의 없다.

시도 2: 짝비교(paired comparison) — 이게 무엇인가

먼저 비유로

키가 비슷한 두 사람 A, B 중 누가 더 큰지를 알고 싶다. 그런데 쓸 수 있는 자가 엉망이라 켈 때마다 영점이 $\pm 2\text{cm}$ 씩 랜덤하게 들어진다.

따로 재기: 오늘 A를 재고(그날 영점 +1.5), 내일 B를 켈다(그날 영점 -0.8). 두 측정값의 차이에는 영점 오차 두 개가 다 섞여 있다. 1cm 차이는 못 잡아낸다.

짝지어 재기: 같은 날, 같은 영점으로 A와 B를 연달아 켈다. 둘 다 +1.5씩 들어졌으니 차이를 구하면 +1.5가 깨끗이 상쇄된다. 자가 엉망이어도 "누가 더 큰지"는 정확히 알 수 있다.

→ 이게 짝비교다. A와 B에 공통으로 들어가는 오차가 있을 때, 차이를 구하면 그게 사라진다.

우리가 기대한 것: A(현재 구성)와 B(휴점 피처를 뺀 것)를 같은 시드로 묶어서 비교하면, "시드 42스러운 공통 편향"이 상쇄되어 차이를 훨씬 정밀하게 켈 수 있을 것이다. 그렇다면 학습 비용을 늘리지 않고도 판별력이 오른다.

실제 결과: 시드 짝 5개로 A와 B를 비교했더니 차이의 $\sigma = 0.0024$ 가 나왔다. 짝짓기가 아무 효과 없었다면 예상되는 값이 약 0.0017인데, 오히려 더 컸다. 이득이 없다.

왜 안 통했나 — 비유가 성립하지 않는 지점

비유에서 "같은 영점"이 성립하려면 오차가 A와 B에 똑같이 얹혀야 한다. 그런데 LightGBM에서 시드는 자의 영점이 아니다.

시드는 "어떤 데이터와 피처를 뽑아 쓸지"를 정한다. 그런데 B는 애초에 피처 하나가 없는 모델이다. 시드가 같아도 뽑히는 피처 조합이 달라지고, 첫 분기부터 다른 트리가 자라나서 결국 완전히 다른 모델이 된다.

→ 자의 영점을 공유하는 게 아니라, 매번 다른 사람이 눈대중으로 재는 것에 가깝다. 같은 사람이 A와 B를 재도 눈대중은 각각 독립적으로 틀린다.

결론: 노이즈를 줄이는 방법은 시드 수를 늘리는 것뿐이고, 그건 비용이 선형으로 는다. 그래서 지금은 스크리닝은 빠르게(시드 2, 7일 간격), 최종 확인만 무겁게(시드 5) 하는 2단 구조로 운영한다.

그리고 제3의 저울이 있다 — 리더보드

제출 횟수가 넉넉하다. CV로 판정이 안 되는 애매한 결과(0.003 근처)는 재실험보다 그냥 제출해서 LB로 확인하는 게 빠르다. Public LB는 우리 CV와 다른 표본으로 만든 독립적인 두 번째 측정이다.

부수효과도 있다 — 쟁점 ④의 CV↔LB 대응 관측치가 아직 1건뿐인데, 제출할 때마다 이게 쌓인다.

쟁점 ④ 우리는 점수를 어떻게 재고 있나 — 3폴드 롤링 오리진

먼저: 왜 자체 검증(CV)이 필요한가

테스트의 정답은 우리에게 없다. 제출해야만 점수가 나온다. 그런데 실험을 하려면 "이 변경이 좋아진 건가?"를 하루에도 수십 번 판단해야 한다. 그래서 학습 데이터 일부를 정답이 있는 척 가려놓고 점수를 재는 장치가 필요하다. 이게 교차검증(CV)이다.

시계열이라 무작위 분할을 쓸 수 없다

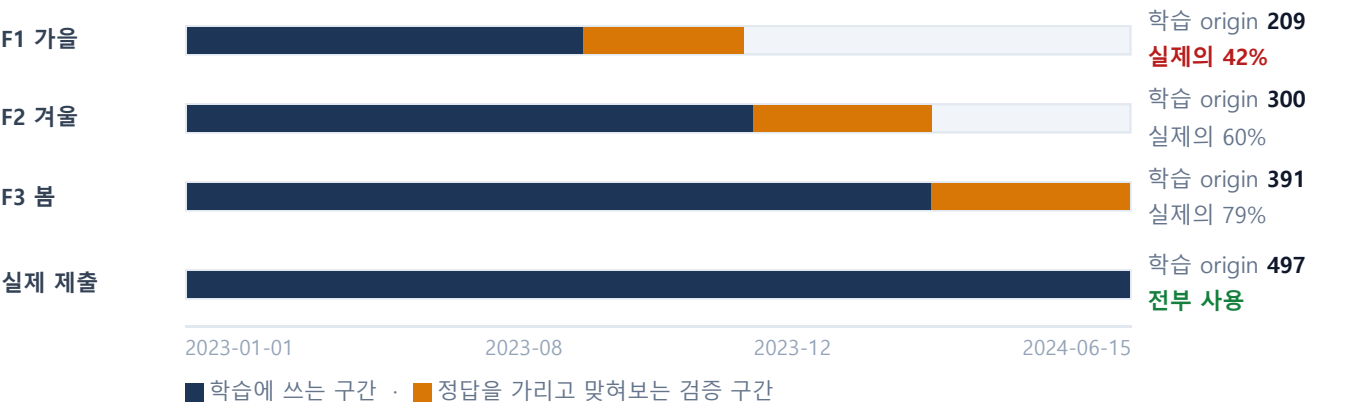
보통 하듯 데이터를 랜덤하게 8:2로 쪼개면 안 된다. 그러면 2024년 데이터로 학습해서 2023년을 맞추는 상황이 생긴다. 실전에서는 절대 불가능한 조건이라, 점수가 실제보다 후하게 나온다(미래 정보 누수).

그래서 롤링 오리진(rolling origin)

대회 구조를 그대로 흉내낸다. 과거의 어떤 날을 "오늘"이라 치고, 그 이전 데이터로만 학습해서, 그 다음 7일을 맞혀보고 채점한다. "오늘"을 조금씩 뒤로 미끄러뜨리며(rolling) 여러 번 반복한다.

"오늘"을 어디에 둘 것인가 — 계절별 3폴드

이 문제는 **계절에 따라 성격이 완전히 다르다**(화담숲은 겨울에 판매량이 정확히 0이다). 겨울 한 구간만 검증하면 겨울에만 잘하는 모델을 뽑게 된다. 그래서 검증 구간을 **가을·겨울·봄 세 개**로 나눴다. 이 셋을 폴드(fold)라 부르고, 그 **평균이 "3폴드 평균"**이다.



여기서 문제가 생긴다

위 그림에서 보이듯, 폴드마다 쓸 수 있는 학습 데이터 양이 다르다. F1(가을)은 2023년 8월 이전 데이터밖에 없어서 실제 제출 상황의 42%뿐이다. F1은 사실상 "데이터가 절반도 안 되는 다른 세상"에서 잤 점수다.

폴드	학습 origin	E03 점수	성격
F1 가을	209	0.6052	저데이터 — 대표성이 없다
F2 겨울	300	0.4461	
F3 봄	391	0.4989	
3폴드 평균	—	0.5167	F1이 끌어내림
F2+F3 평균	—	0.4725	실제 LB와 0.009 차이
실제 제출 (LB)	497	0.4818	← 우리가 예측하고 싶은 값

3폴드 평균 0.5167은 실제 LB 0.4818보다 0.035나 비관적이다. 반면 F2+F3만 평균내면 0.4725로 LB와 0.009 차이밖에 안 난다.

왜 이게 중요한가 — 실제로 결론이 뒤집힌 사례

item_id를 빼야 하나?

설정	F1(저데이터)	F2	F3	3폴드 평균	F2+F3
현재 구성	0.5946	0.4572	0.4956	0.5158	0.4764
item_id 제거	0.5692	0.4612	0.5071	0.5125 좋아 보임	0.4842 실제론 악화

3폴드 평균만 보면 "item_id를 빼면 개선"으로 읽힌다. 그런데 폴드별로 뜯어보면 **개선된 건 F1 하나뿐**이고 F2·F3는 오히려 나빠졌다. F1의 큰 개선(0.5946→0.5692)이 평균을 끌고 간 것이다.

왜 F1에서만 좋아졌나: item_id는 "이게 몇 번 품목인지"를 알려주는 피처다. 데이터가 충분하면 품목별 특성을 배우는 데 유용하지만, **데이터가 적으면 각 품목의 몇 안 되는 사례를 통째로 외워버린다**(과적합). 즉 **부호가 데이터량에 따라 뒤집히는 피처**다.

실제 제출은 학습 origin이 497개로 F3(391)보다도 많다. **유지가 맞다**.

운영 규칙

채택 여부 판정은 항상 **F2+F3**로 한다(이 기준의 2σ 는 0.0034). F1은 "저데이터 환경에서 버티는가" **스트레스 테스트**로 따로 읽는다. 실험 간 단순 상대비교는 3폴드 평균으로 해도 무방하다(같은 조건끼리 비교라서).

단, CV↔LB 대응은 **관측이 아직 1건뿐**이고 Public LB는 테스트의 50% 샘플이라 노이즈가 있다. 제출을 쌓으면서 계속 확인해야 한다.

4. 우리 모델은 무엇을 보고 있나 — 피처 57개

4.1 학습 데이터 한 줄이 무엇인지부터

모델에게 던지는 **한 줄(샘플)**은 이런 질문이다:

"**담하_공깃밥**이라는 품목이 있다. 지난 28일간 이 품목은 [12, 0, 15, 18, ...]개씩 팔렸다. 이 영업장 전체는 [340, 0, 410, ...]개씩 팔렸다. 맞춰야 할 날은 **3일 뒤 토요일**이고 공휴일은 아니며 화답숲은 개장 중이다. 이 품목은 **가격 등급 1의 한식**이다.
→ **그래서 그날 몇 개가 팔릴까?**"

피처란 이 질문에 들어가는 **숫자 하나하나**다. 우리는 지금 57개를 넣고 있다. 아래가 그 전부다.

4.2 피처 전체 목록 — 7개 그룹

① win — 창 통계 14개 "이 품목이 최근 28일간 어땠나"

창(28일) 안의 **내 품목 판매량 28개** 숫자를 여러 방식으로 요약한 것이다. 가장 기본이고, **단독으로 써도 가장 강력하다**(0.569).

w_mean · w_std · w_max · w_median	28일 전체의 평균 / 표준편차 / 최댓값 / 중앙값
w_nzratio	28일 중 0이 아닌 날의 비율 — 얼마나 자주 팔리는 품목인가
w_posmean · w_posmedian	0인 날을 뺀 양수만의 평균 / 중앙값 — "팔린 날엔 얼마나 팔리나"
w_last	바로 어제(창의 마지막 날) 판매량
w_last7 · w_last14	최근 7일 / 14일 평균
w_prev7	그 이전 7일(8~14일 전) 평균
w_trend	$w_last7 \div w_prev7$ — 최근 일주일 이 그 전 주보다 늘었나 줄었나
w_days_since	마지막으로 팔린 게 며칠 전인가 — 간헐 품목 판별용
w_last7_nz	최근 7일 중 팔린 날의 비율

② dow — 같은 요일 통계 5개 "맞혀야 할 요일엔 어땠나"

맞혀야 할 날이 토요일이면, **창 안에 있는 토요일 4개**만 골라서 요약한다. 요일 효과가 강한 리조트라 중요하다.

d_mean · d_max · d_posmean	그 요일들의 평균 / 최댓값 / 양수만의 평균
d_nzratio	그 요일 중 팔린 비율
d_last	가장 최근 그 요일 의 판매량 (= 지난주 같은 요일)

표본이 **4개뿐**이라 노이즈가 크다는 게 약점이다. 이걸 보완하려던 시도가 4.4절의 요일 프로파일이다.

③ closed — 휴점 보정 6개 "평균이 휴점 때문에 왜곡됐나"

영업장 전체 합계가 0인 날을 **휴점일**로 정의하고, 그 날들을 건너낸 통계를 따로 준다. 2.3⑥에서 설명한 그것이다.

st_closed_ratio	창 28일 중 휴점일 비율
st_closed_last7	최근 7일 중 휴점일 비율
w_mean_open	영업일만 으로 계산한 평균 ← 왜곡되지 않은 진짜 평균
w_nzratio_open	영업일 중 팔린 비율
d_mean_open	같은 요일 중 영업했던 날만 의 평균
st_days_since_open	마지막 영업일이 며칠 전인가 — 휴장 중인지 판별

④ cross — 교차 품목 8개 "오늘 이 식당에 사람이 얼마나 왔나"

지금까지는 전부 **내 품목** 얘기였다. 학습 행 하나에는 자기 품목 통계만 들어 있어서, **같은 영업장의 다른 품목이 얼마나 팔렸는지는 모델이 구조적으로 알 수 없다**. 그걸 넣어주는 그룹이다.

x_store_last7 · x_store_prev7	영업장 전체 합계 의 최근 7일 / 이전 7일 평균
x_store_trend	영업장 전체의 추세 (최근7 ÷ 이전7)
x_store_dow	영업장 전체의 같은 요일 평균
x_proxy_last7 · x_proxy_dow	인원수 프록시 품목 의 최근 7 / 같은 요일 평균 (아래 설명)
x_share_win · x_share_dow	침투율 = 내 품목 ÷ 영업장 합계. 창 전체 / 같은 요일 기준

"**인원수 프록시**"란 그 식당의 방문 인원수를 가장 잘 대변하는 품목이다. 어떤 메뉴는 '선택'이 아니라 '인원수'를 따라가기 때문이다. 영업장마다 자기 자신을 뺀 나머지 합계와 상관성이 가장 높은 품목을 데이터로 골랐다.

영업장	선정된 프록시	상관	판매순위	해석
포레스트릿	치즈 핫도그	0.978	4/12	스낵바 — 대표 메뉴가 곧 인원수
화담숲주막	해물파전	0.971	1/8	
담하	공깃밥	0.687	1/42	한식당이라 밥이 인원수를 따라간다
느티나무BBQ	콜라 (단체)	0.883	5/23	음료가 인원수 지표
미라시아	브런치(어린이)	0.600	5/31	가족 단위 방문이 지배적 이라는 뜻
연회장	Grand Ballroom	0.274	15/23	상관이 낮다 — B2B라 행사마다 제각각

※ 처음엔 '영업장 총합과 상관이 최대'인 품목을 골랐다가 **순환 논리 결함**을 발견했다 — 총합에 자기 자신이 포함되니 그냥 **제일 많이 팔리는 메뉴**가 뽑혔던 것이다. 자기를 뺀 나머지와 상관을 재도록 수정했다.

⑤ **ctx** — 맥락 4개

sd_lastweek	정확히 7일 전 같은 날의 판매량 (창 통계와 별개로 직접 준다)
st_mean · st_nz	영업장 전체의 창 평균 / 0이 아닌 비율
horizon	며칠 뒤를 맞히는가 (1~7). 먼 날일수록 불확실하다는 걸 모델이 알아야 한다

⑥ **cal** — 달력 · 도메인 13개 "그날이 어떤 날인가"

맞혀야 할 **미래 날짜**에서 뽑는다. 미래지만 **달력은 이미 확정된 사실**이라 누수가 아니다.

dow · month · day	요일 / 월 / 일
is_weekend	토·일인가
is_holiday · is_holiday_eve	공휴일인가 / 공휴일 전날인가 (2023~2025 대체공휴일 포함 수동 입력)
is_dayoff	주말 또는 공휴일 = 쉬는 날인가
doy_sin · doy_cos	연중 위치를 사인/코사인 한 쌍 으로. 12월 31일과 1월 1일이 이어지게 만드는 표준 기법
hwadam_open	화담숲 개장기(3/20~11/30)인가
ski_season · ski_peak	스키 시즌(12~2월 + 3/5까지) / 성수기 구간(12/20~1/31)인가
foliage	단풍 시즌(10/15~11/20)인가

이 그룹은 **단독으로 쓰면 1.0865로 최악**인데(날짜만 알고 품목을 모르면 예측이 불가능하니 당연하다), **빼면 손실이 가장 크다(+0.0173)**. 상호작용 피처의 교과서적 사례다.

⑦ **item** — 품목 고정 속성 7개 "이게 뭐 하는 품목인가"

store · cluster	어느 영업장인가 / 어느 군집인가(화담숲·스키·그린·상시·B2B)
category · season_hint	메뉴 종류 / 계절 성향 (웹 리서치로 만든 라벨)
item_id	품목 고유 번호 — gain 40.9%로 1위 지만 쟁점 ④의 함정이 있다
price	가격 5등급 (3천/8천/2만/5만원 경계)
is_high_weight	담하·미라시아인가 (채점 가중치 높은 업장)

4.3 그룹별로 얼마나 중요한가

피처 그룹을 **하나씩 빼보고(ablation)** 점수가 얼마나 나빠지는지 쟀다. 2σ=0.003을 넘는 것만 확실한 결론이다.

그룹	빼면	그것만 쓰면	해석
cal 달력	+0.0173	1.0865 (최악)	단독 최악 · 조합 최고. 품목을 모르면 날짜만으론 예측 불가
요일 정보 전부	+0.0172	—	요일 자체가 결정적
win 창 통계	+0.0109	0.5690 (최고)	단독으로도 강하고 대체 불가. "최근 4주가 정보의 대부분"
item_id	+0.0068	—	유지 필수 (F2+F3 기준)
closed 휴점	+0.0036	—	유지 필수
공휴일	+0.0031	—	경계선 (거의 유의)
도메인 플래그	+0.0011	—	차이 없음 — 그러나 뺄 이유도 없다
ctx · dow	±0.001	—	차이 없음 (다른 그룹과 중복되는 듯)

gain 중요도는 실제 기여가 아니다

LightGBM이 알려주는 "중요도(gain)"에서 item_id 가 **40.9%로 압도적 1위**다. 그런데 실제로 빼보면 기여는 그에 훨씬 못 미치고, 저데이터 폴드에서는 **오히려 해롭다**.

gain은 "모델이 그 피처를 얼마나 자주 크게 썼나"이지 "그게 있어서 얼마나 좋아졌나"가 아니다. **빼봐야 (ablation) 안다**.

4.4 넣어봤다가 뺀 것 — 그리고 왜

prof 품목별 요일 프로파일 5개 — 기각(+0.0014)

동기: 엑셀을 정독하다 이런 패턴을 발견했다 — '단체' 메뉴는 목·금에 몰리고 일반 메뉴는 토·일에 몰린다. 요일의 의미가 품목마다 정반대인 것이다.

구분	월	화	수	목	금	토	일
단체성 메뉴 (34개)	0.80	0.88	0.99	1.55	1.55	0.83	0.40
일반 메뉴 (159개)	0.89	0.73	0.67	0.82	1.08	1.63	1.18

가설: dow 그룹은 창 안 같은 요일이 **4개뿐**이라 노이즈가 커서 이 패턴을 못 잡을 것이다. 학습기간 **전체**로 안정적인 요일 프로파일을 만들어 정적 피처로 주자.

결과: 피처는 관찰을 정확히 재현했다(연회장 Regular Coffee 목요일 2.5배 등 — 기업 워크숍이 목요일에 몰린다는 해석과 일치). **그런데 점수는 +0.0014로 나빠졌다**(5회 중 4회 악화).

왜: 모델이 item_id × dow 로 이미 만들 수 있던 정보였다. 학습 행이 32만 줄이라 그 정도 상호작용은 트리가 스스로 학습한다. 게다가 피처가 늘면 feature_fraction=0.85 때문에 **기존의 좋은 피처가 뽑힐 확률이 희석된다**.

코드는 지워지지 않고 남아 있다(PROF_KEYS). 모델 구조가 바뀌면 값어치가 생길 수도 있다.

4.5 아직 안 해본 피쳐 — 후보 목록

기존에 적어둔 것

후보	내용과 기대
ICE/HOT 비율 = 기온 프록시	창 28일간 아이스 음료 ÷ 뜨거운 음료 비율. 날씨 데이터가 금지돼 있으니 합법적 대체안 이다. 사람들이 뭘 마셨는지가 그날이 더웠는지 알려준다. 조건 ㉔ 만족 가능성 — 다만 창 안 다른 품목 정보라 cross 와 겹칠 수 있다
온·냉 메뉴 구분	지금 아메리카노 HOT과 ICE가 둘 다 그냥 '음료' 로 분류돼 있다. 계절 방향이 정반대인데 같은 범주다. 라벨만 나누면 되는 싼 작업
품목 클러스터링	판매 패턴으로 k-means. 다만 <code>item_id</code> 가 이미 있어서 중복 가능성이 높다
계층 예측용 침투율 모델	영업장 총량을 먼저 예측하고 × 침투율로 품목을 내는 구조. 피쳐가 아니라 모델 구조 변경 에 가깝다 → 5장 순위 3
발생확률 주입	"이 품목이 그날 팔릴 확률"을 별도 모델로 내서 피쳐로 넣는다. 쟁점 ①과 모순돼 보이지만 다르다 — 확률을 예측 대상으로 삼는 게 아니라 수량 예측의 힌트로 쓰는 것이다 . 팔릴 확률이 낮은 날은 조용한 날이고, 조용한 날은 팔려도 적게 팔린다

추가 제안 — 4.6절의 원칙을 통과하는 것들

후보	왜 값어치가 있을 만한가
전환일까지 남은 일수 (가장 유망)	지금 <code>hwadam_open · ski_season</code> 은 참/거짓 플래그뿐이다. "개장 3일 전"과 "개장 후 두 달"이 같은 값으로 들어간다. 개장·폐장까지 며칠 남았는 지를 숫자로 주면 전환 구간의 급변을 표현할 수 있다. 근거 : 오차 1~3위가 전부 계절 매장이고, 테스트 10개 중 3개가 전환 주간 (TEST_04/07/08)이다. 게다가 창은 28일뿐이라 모델이 창만 보고는 절대 알 수 없는 정보 다 → 조건 ㉔ 명확히 만족
연휴 길이 · 연휴 내 위치	지금은 <code>is_holiday · is_holiday_eve</code> 뿐이다. 3일 연휴의 첫날 / 가운데 / 마지막날 은 리조트 수요가 전혀 다르다(입실·체류·퇴실). 달력에서 계산 가능하고 합법 이며, 창 밖 정보라 트리가 스스로 만들 수 없다
영업 재개 직후 여부	<code>st_days_since_open</code> 은 있지만 "휴장이 길었다가 막 다시 연 첫 주 "라는 특수 상태를 직접 표현하진 않는다. 화담숲처럼 3개월 통째로 쉬는 곳은 재개 직후 패턴이 평상시와 다를 가능성이 있다
간헐성의 규칙성	<code>w_nzratio</code> 는 "얼마나 자주 팔리나"인데, " 규칙적으로 주 1회 "와 " 불규칙하게 4번 "을 구분하지 못한다. 판매 간격의 표준편차를 주면 구분된다. 다만 창 표본이 적어 노이즈 우려

단, 기대치를 낮게 잡을 것

피쳐 추가는 3연속 시도에서 **확실한 성공이 1건뿐**이었다. 그리고 기대 개선폭이 0.000~0.005라면 **측정 한 계(0.003)에 걸쳐 성공해도 판정이 안 된다**. 그래서 5장에서 **우선순위 4**위다.

다만 **비용이 싸다**(피쳐 몇 줄 추가). 위 후보들은 묶어서 **한 번에 테스트**하고, 애매하면 **그냥 제출해서 LB로 확인**하는 게 효율적이다.

4.6 세 번의 시도에서 얻은 원칙

시도	결과	왜
휴점 보정	-0.0036	창 통계가 실제로 왜곡되던 것을 바로잡음
요일 프로파일	+0.0014	모델이 이미 알 수 있던 정보의 재표현
교차 품목	-0.0010	구조적으로 새 정보이나 <code>st_mean</code> 과 부분 중복

원칙

"사람이 발견한 규칙"을 떠먹이는 것보다 "데이터가 왜곡된 지점을 바로잡는 것"이 이득이 크다.

트리는 상호작용을 스스로 찾는다. 그러므로 새 피처가 값어치를 가지려면 둘 중 하나여야 한다:

- ㉠ 모델이 구조적으로 접근 불가능한 정보인가? (창 밖 달력, 다른 품목 판매량 등)
- ㉢ 데이터 왜곡을 바로잡는 것인가? (휴점일이 평균을 희석하던 문제 등)

둘 다 아니면 대개 중복이고, 피처가 늘어난 만큼 기존 좋은 피처가 희석되어 오히려 손해다.

부수 발견 — 알아둘 만한 것

트리는 단조변환에 불변이다. `log(price)` 를 넣은 결과가 원본 가격과 소수점 4자리까지 동일했다. 트리는 값의 크기 순서만 보기 때문이다. → 로그 변환·정규화·스케일링은 트리에 무의미하고, 값을 묶는 범주화만 의미가 있다. (동시에 파이프라인이 결정론적이라는 확인도 됐다.)

피처 생성이 100배 빨라졌다. `_window_stats` 의 품목별 중앙값 계산을 파이썬 루프(9.6만 회)에서 `nanmedian` 벡터화로 교체 — 5분 → 3초. 실험 회전 속도가 곧 실험 개수다.

5. 앞으로 할 일 — 순서와 기대 효과

지금까지는 피처만 손냈다. 그런데 남은 영역 중 기대값이 큰 건 다른 쪽이다.

순위	작업	기대 개선	확신도	왜 이 순서인가
1	Phase 5-a 하이퍼파라미터 탐색	0.005 ~0.010	높음	사실상 거의 안 해봤다. num_leaves와 rounds 두 개만 건드렸고 그것만으로 0.004를 얻었다. learning_rate · min_data_in_leaf · feature_fraction · 정규화는 기본값 그대로다. 가장 싸고 가장 확실한 남은 이득
2	Phase 5-b XGBoost · CatBoost 추가 + 알고리즘 앙상블	0.005 ~0.015	높음	지금 앙상블은 LightGBM 시드 5개뿐이다. 같은 알고리즘이라 편향을 공유한다. 알고리즘이 다르면 오차의 상관이 낮아 평균의 이득이 훨씬 크다. 보너스: 앙상블은 쟁점 ③의 시드 노이즈 자체를 줄여줘서 이후 모든 실험의 판별력이 올라간다
3	Phase 5-c 모델 구조 horizon별 7모델 / 세그먼트 분리 / 계층 예측	0.005 ~0.015	중간	오차 분해가 구체적인 근거를 준다 — +1일 0.498 vs +6일 0.577(horizon별 분리 근거), 계절매장 0.62~0.71 vs 상시매장 0.50~0.56(세그먼트 분리 근거). 다만 모델 수가 늘면 학습 비용도 배로 늘고, 세그먼트를 쪼개면 각 모델의 학습 데이터가 줄어 쟁점 ④의 저데이터 함정을 다시 만난다
4	4.5절의 신규 피처	0.000 ~0.005	낮음	4.6절 원칙대로 기대값이 낮다. 게다가 기대 크기가 측정 한계(0.003)에 걸쳐 있어 이득이 있어도 판정이 안 된다. 다만 비용이 싸므로 묶어서 한 번에 테스트하고 애매하면 제출로 확인
—	타깃-손실 설계 / 후처리	?	낮음	log1p+L1은 이미 대안(l2+huber)을 이겼고, 후처리는 하한 1.0이 이론적 최적이다. 새로 팔 게 별로 없다

기대 개선폭을 더하지 말 것

위 숫자들은 서로 겹친다. 다 합치면 0.03이지만 실제로는 그렇게 안 나온다. 예를 들어 하이퍼파라미터를 잘 맞춘 모델일수록 앙상블에서 추가로 얻는 이득이 줄어든다.

현실적 전망: 0.46~0.47. 1등 0.44까지는 어려울 것으로 본다. 다만 55등 → 상위권 이동은 충분히 기대할 수 있다.

우선순위를 정한 두 번째 축 — "젤 수 있는가"

기대 크기만으로 순서를 정한 게 아니다. **측정 가능성**도 같이 봤다.

우리 저울의 눈금은 0.003이다(쟁점 ③). 기대 개선이 0.000~0.005인 작업은 **성공해도 성공한 줄 모른다**. 반대로 0.005~0.015짜리 작업은 확실히 판정된다. 그래서 "큰 것부터"가 곧 "젤 수 있는 것부터"이기도 하다.

순위 2(알고리즘 앙상블)가 특히 좋은 이유가 여기 있다 — 점수를 올리면서 동시에 저울의 눈금을 촘촘하게 만든다. 그 다음 작업들이 더 쉬워진다.

실험 운영 규약 (전원 동일 적용)

항목	규칙
판정 지표	F2+F3 평균 ($2\sigma = 0.0034$). F1은 저데이터 스트레스 테스트로 별도 판독
채택 기준	평균 0.003 이상 개선 AND 시드 조합 5회 중 4회 이상 같은 방향
스크리닝 설정	leaves 127 · rounds 600 · 시드 2 · origin 7일 간격 (빠른 비교용)
최종 확인	leaves 255 · rounds 1000 · 시드 5
제출	아끼지 않는다. 횟수가 넉넉하다. CV로 판정이 안 되는 애매한 결과는 재실험보다 제출이 빠르다. 제출할 때마다 CV↔LB 보정 데이터도 쌓인다
기록	모든 실험은 experiments/log.md 표에 남긴다. 실패한 실험도 반드시 — 재시도 금지 목록이 곧 시간 절약이다

6. 미팅에서 정할 것

- 로그 불일치 3건 확인. 문서화 과정에서 발견됐다.
 - log.md 는 "가격을 categorical로 전환"이라 기록했으나 **실제 코드는 순서형 숫자로 되돌려져 있다** (features.py: categorical로 바꾸니 0.5159 → 0.5186으로 악화). 어느 쪽이 최종인지 확정하고 로그를 고쳐야 한다. ※ 어차피 가격 범주화의 이득(-0.0012)은 2σ 를 못 넘어 입증되지 않은 상태다.
 - Phase 4-d가 본문엔 "진행 중", 체크리스트엔 완료로 되어 있다. 저장된 결과는 **짜비교뿐**이고 "시드 수를 늘리면 σ 가 $1/\sqrt{n}$ 로 주는가"의 결과는 **어디에도 남아 있지 않다.** 다시 돌릴지 결정 필요
 - 헤더 날짜가 07-30인데 4-e/4-f는 07-31에 돌아왔다
- Phase 5-a의 탐색 예산.** 전수 그리드는 비용이 크다. 어떤 파라미터를 몇 개 값까지 볼지, 스크리닝 설정으로 몇 회까지 돌릴지 정해야 한다.
- 스크리닝 시드 수 고정.** 짜비교가 실패했으니 노이즈를 줄이려면 시드를 늘리는 수밖에 없는데 비용이 선형으로 는다. 지금 2개인데 몇으로 같지 — **판별력과 실험 회전 속도의 맞교환**이다.
- 작업 분담.** 순위 1(하이퍼파라미터)과 순위 2(다른 알고리즘)는 **서로 독립이라 동시에 진행 가능하다.** 순위 3(모델 구조)은 1이 끝난 뒤가 낫다(좋은 파라미터를 물려받아야 공정한 비교가 된다).

곤지암리조트 식음업장 수요예측 (Dacon 236559 · LG Aimers 7기) — 팀 브리핑 · 2026-08-01 작성
근거 자료: experiments/log.md, phase2_results.json, phase4a~4f 실험 결과,
src/features.py · validate.py · config.py